

1) **Диалоговые программы. Понятие дружелюбности.**

Диалоговая программа — это программа, которая взаимодействует с пользователем, принимая ввод данных и выводя результаты. Такие программы строятся по принципу диалога: программа задаёт вопросы (выводит подсказки), пользователь вводит ответы. Пример: запрос возраста у пользователя через *cin*, а затем вывод результата через *cout*. Понятие дружелюбности означает удобство и понятность программы: корректные и вежливые сообщения, понятная логика работы, обработка ошибок ввода. Например, если пользователь ввёл не число, программа должна это корректно обработать и предложить повторить ввод.

2) **Условный оператор и условная операция.**

Условный оператор в C++ — это конструкция *if*, которая позволяет выполнить определённый блок кода, если условие истинно. Пример:

```
if (x > 0) cout << "Число положительное";
```

Также можно использовать *else* для альтернативного действия, если условие ложно. Кроме того, в C++ существует условная операция, реализуемая тернарным оператором *?:*, которая возвращает одно из двух значений в зависимости от условия.

Пример:

```
int max = (a > b) ? a : b;
```

Здесь переменной *max* будет присвоено большее из двух значений — *a* или *b*. Это краткая форма записи простой конструкции *if...else*.

3) **Операции отрицания и побитового дополнения.**

В C++ логическая операция отрицания обозначается как *!* и используется для инверсии булевого значения: *!true → false*, *!false → true*.

Пример:

```
bool x = false;
```

```
if (!x) cout << "x равно false";
```

Побитовая операция дополнения *~* инвертирует каждый бит целого числа.

Пример:

```
int a = 5; // двоично: 00000101
```

```
int b = ~a; // 11111010 → -6 (в доп. коде)
```

Эта операция полезна при низкоуровневой обработке данных, работе с флагами, битовыми масками и т. п.

4) Логические операции.

Логические операции в C++ выполняются над булевыми значениями (*true* или *false*) и используются в условных выражениях:

- && (логическое И): результат — *true*, если оба операнда истинны.
- || (логическое ИЛИ): результат — *true*, если хотя бы один из операндов истинный.
- ! (логическое НЕ): инвертирует значение.

Пример:

```
int a = 5, b = 10;  
if (a > 0 && b > 0) cout << "Оба числа положительные";
```

Важно: && и || — короткозамыкающиеся операции. Это значит, что, если результат можно определить по первому операнду, второй не вычисляется.

5) Побитовые операции.

Побитовые операции работают на уровне отдельных битов целых чисел:

- & — побитовое И: бит равен 1 только если оба соответствующих бита равны 1.
- | — побитовое ИЛИ: бит равен 1, если хотя бы один из битов равен 1.
- ^ — исключающее ИЛИ (XOR): бит равен 1, если биты различны.
- ~ — побитовое НЕ: инвертирует все биты.
- << — сдвиг влево: умножение на 2^n .
- >> — сдвиг вправо: деление на 2^n (с потерей младших битов).

Пример:

```
int x = 5;        // 00000101  
int y = 3;        // 00000011  
cout << (x & y); // 00000001 → 1  
cout << (x | y); // 00000111 → 7  
cout << (x ^ y); // 00000110 → 6
```

Побитовые операции часто используются в системном программировании и для работы с флагами.

6) **Изменение типа выражения.**

В C++ можно преобразовывать (кастовать) один тип данных в другой. Это нужно, например, при делении двух *int*, когда нужен *float* результат. Пример:

```
int a = 5, b = 2;
float result = (float)a / b; // 2.5, иначе будет 2
float result = static_cast < float > (a/b);
```

Существует явное преобразование (через (тип) или *static_cast* (*static_cast*<новый_тип>(выражение))) и неявное, когда компилятор автоматически приводит типы (например, *int* к *float* в арифметике). Явное приведение лучше видно и контролируемо.

7) **Литералы в языке Си.**

Литералы — это фиксированные значения в коде: числа, символы, строки и булевы значения:

- Числовые: 42, 3.14, 0xFF (шестнадцатеричный), 075 (восьмеричный).
- Символьные: 'A', '\n'.
- Строковые: "Hello" — массив *char*.
- Булевы: *true*, *false*.

Также можно указывать суффиксы:

```
long l = 100L;    // long
float f = 3.14f;  // float
```

Литералы важны для понимания типов и контроля переполнений.

8) **Операции "адрес" и косвенной адресации (разадресации).**

В C++ можно работать с указателями — переменными, хранящими адреса других переменных.

- & — операция взятия адреса:

```
int x = 10;
int * p = &x; // p хранит адрес x
```

- * — операция разыменования (косвенная адресация):

```
cout << * p; // получаем значение, на которое указывает p
```

Эти операции необходимы при передаче данных по ссылке, динамической памяти и работе с массивами.

9) **Структура объявления типов. Классы памяти.**

В языке C/C++ тип переменной указывается при объявлении:

int x;

float y;

Структура объявления типа может включать модификаторы:

unsigned long int a;

Классы памяти определяют область действия и время жизни переменной:

- *auto* (по умолчанию) — локальные переменные.
- *register* — попытка разместить переменную в регистре.
- *static* — сохраняет значение между вызовами.
- *extern* — указывает, что переменная определена в другом файле.

Пример:

static int counter = 0;

Эта переменная сохраняет своё значение между вызовами функции.

10) **Классификация типов данных с размерами в памяти (в байтах).**

Типы данных в C++ классифицируются на:

- Целые:

1. *char* (1 байт)
2. *short* (2 байта)
3. *int* (4 байта)
4. *long* (4–8 байт)
5. *long long* (8 байт)

- С плавающей точкой:

1. *float* (4 байта)
2. *double* (8 байт)
3. *long double* (10–16 байт, зависит от компилятора)

Булевы:

1. *bool* (1 байт, но может занимать меньше)

Модификаторы:

1. *signed* / *unsigned* — задают знак числа

Размеры могут отличаться на разных архитектурах, но обычно можно уточнить через *sizeof()*.

11) Подпрограммы в языке C++. Определение функции. Возврат значений.

Подпрограмма — это отдельный блок кода (функция), который выполняет конкретную задачу и может возвращать результат. В C++ функция определяется так:

```
тип_возврата имя(параметры) {  
    // тело функции  
    return значение; // если тип не void  
}
```

Пример:

```
int sum(int a, int b) {  
    return a + b;  
}
```

Функция вызывается по имени с аргументами: `int s = sum(2,3);`. Возврат значения через `return` передает результат обратно вызывающей части программы.

12) Передача в подпрограмму массивов.

Массивы в C++ передаются в функции как указатели на первый элемент.

Пример:

```
void printArray(int arr[], int size) {  
    for (int i = 0; i < size; i++)  
        cout << arr[i] << " ";  
}
```

Вызов:

```
int a[3] = {1, 2, 3};  
printArray(a, 3);
```

При передаче массива фактически передается адрес, поэтому внутри функции можно изменять содержимое массива.

13) Объявление, вызов функции и возврат значений.

Объявление функции (прототип) сообщает компилятору о ее наличии:

```
int multiply(int, int);
```

Определение содержит тело функции:

```
int multiply(int a, int b) {  
    return a * b;  
}
```

Вызов:

```
int res = multiply(4, 5);
```

Возврат значения осуществляется оператором `return`. Если функция не возвращает значения, используется `void`.

14) **Работа с файлами.**

В C++ для работы с файлами используются библиотеки `<fstream>`, классы `ifstream` (чтение), `ofstream` (запись), и `fstream` (чтение и запись).

Запись:

```
#include <fstream>
ofstream fout("file.txt");
fout << "Привет, файл!\n";
fout.close();
```

Чтение:

```
ifstream fin("file.txt");
string line;
getline(fin, line);
cout << line;
fin.close();
```

Важно открывать и закрывать файлы, проверять успешность открытия.

15) **Вызов функций с переменным числом аргументов.**

Функции с переменным числом аргументов используют `<cstdarg>`.

Пример:

```
#include <cstdarg>
int sum(int count,...) {
    va_list args;
    va_start(args, count);
    int total = 0;
    for (int i = 0; i < count; i++)
        total += va_arg(args, int);
    va_end(args);
    return total;
}
```

Вызов:

```
int s = sum(3, 10, 20, 30); // 60
```

Первый параметр — количество аргументов, после идут сами значения. Это способ реализовать функции как `printf`.

16) **Возврат из функции нескольких значений.**

В C++ функция может возвращать только одно значение, но есть несколько способов вернуть несколько результатов:

- Использование выходных параметров через указатели или ссылки:

```
void divide(int a, int b, int& quotient, int& remainder) {  
    quotient = a / b;  
    remainder = a % b;  
}
```

- Возврат структуры или класса, содержащей несколько полей:

```
struct Result {  
    int quotient;  
    int remainder;  
};  
Result divide(int a, int b) {  
    return {a / b, a % b};  
}
```

17) **Структура программы и время жизни переменных, объявленных на внутреннем уровне.**

Внутренние (локальные) переменные объявляются внутри функций или блоков {}. Их время жизни — от момента входа в блок до выхода из него. При выходе память освобождается.

Пример:

```
void func() {  
    int x = 10; // локальная переменная  
} // здесь x уничтожается
```

По умолчанию локальные переменные — автоматические (*auto*), их значения не сохраняются между вызовами функции, если не объявлены как *static*.

18) **Общая структура программы, состоящей из нескольких файлов. Локальные и глобальные переменные. Переменные, видимые в нескольких файлах.**

Большие программы разбиваются на несколько файлов *.cpp* и *.h*.
- Глобальные переменные объявляются вне функций, видны во всем файле и (если объявлены с *extern*) доступны из других файлов.
- Локальные переменные видны только в функции или блоке, где объявлены.
Пример:

```
// file1.cpp
int global_var = 10;
Место для уравнения. // file2.cpp
extern int global_var;
```

Так можно разделять код и использовать общие данные.

19) **Время жизни и области видимости функций и переменных, объявленных на внешнем уровне.**

Переменные и функции, объявленные на внешнем уровне (вне всех функций), имеют глобальную область видимости — доступны из всего файла. Если объявлены с *extern*, доступны и из других файлов. Время жизни глобальных переменных — с запуска программы до её завершения. Функции видимы везде, где есть их объявление (прототип).

20) **Типы, определяемые пользователем. Структуры.**

В C++ можно создавать свои типы данных через *struct* и *class*. Структура — это набор связанных данных разных типов:

```
struct Point {
    int x;
    int y;
};
```

Создаём переменную: *Point p = {10, 20};*.
Структуры полезны для объединения данных и упрощения передачи связанных параметров.

21) Объединения.

Объединение (*union*) — это специальный пользовательский тип данных, позволяющий хранить в одной области памяти разные типы, но только одно значение в момент времени. Все поля *union* занимают одну и ту же память.

```
union Data {  
    int i;  
    float f;  
    char c;  
};
```

Размер *union* равен размеру самого большого поля. Используется для экономии памяти или работы с низкоуровневыми структурами.

22) Поля битов. битов.

Поля битов (*bit – fields*) — это возможность определить в структуре переменные, занимающие не полный байт, а определённое число бит.

Пример:

```
struct Flags {  
    unsigned int flag1 : 1;  
    unsigned int flag2 : 3;  
    unsigned int flag3 : 4;  
};
```

Так можно экономить память, хранить компактно логические и маленькие целочисленные значения.

23) Динамическая память. Резервирование, освобождение памяти в языке С.

В языке С динамическая память выделяется функцией *malloc* и освобождается *free*.

Пример:

```
int * p = (int *)malloc(sizeof(int) * 10); // выделить массив из 10 int  
// использование  
free(p); // освобождение памяти
```

Если не освободить память — возникнет утечка. *malloc* возвращает *NULL*, если память не выделена.

24) **Динамическая память. Резервирование, освобождение памяти в языке C++.**

В C++ для динамического выделения используется оператор *new*, для освобождения — *delete*.

Пример:

```
int * p = new int[10]; // выделение массива из 10 int
// использование
delete[] p; // освобождение памяти
```

Для одиночного объекта:

```
int * x = new int(5);
delete x;
```

new выбрасывает исключение при ошибке выделения, или можно использовать *nothrow*.

25) **Связные динамические структуры. Списки. Двусвязные списки.**

Связный список — структура данных, состоящая из узлов, каждый из которых содержит данные и указатель(и) на следующий (и предыдущий) узлы.

- Односвязный список: каждый узел ссылается только на следующий.

- Двусвязный список: каждый узел содержит указатели и на следующий, и на предыдущий.

Пример узла двусвязного списка:

```
struct Node {
    int data;
    Node * next;
    Node * prev;
};
```

Связные списки позволяют динамически изменять размер и эффективно вставлять/удалять элементы.

26) **Связные динамические структуры. Очередь. Односвязные списки.**

Очередь — структура данных, работающая по принципу *FIFO* (первым пришёл — первым вышел). Обычно реализуется на базе односвязного списка с указателями на начало и конец. Односвязный список — последовательность узлов, где каждый содержит данные и указатель на следующий элемент. Пример добавления элемента в очередь (*enqueue*):

```
struct Node {  
    int data;  
    Node * next;  
};
```

Место для уравнения. *Node * front = nullptr; // начало*

*Node * rear = nullptr; // конец*

Место для уравнения. *void enqueue(int val) {*
 *Node * newNode = new Node{val, nullptr};*
 if (rear) rear->next = newNode;
 else front = newNode;
 rear = newNode;
}

Очередь используется для упорядоченного хранения данных.

27) **Связные динамические структуры. Кольцевые списки.**

Кольцевой список — разновидность связного списка, где последний узел ссылается на первый, образуя кольцо. Может быть односвязным или двусвязным.

Пример:

```
struct Node {  
    int data;  
    Node * next;  
};
```

// последний элемент указывает на первый

Используется для циклической обработки элементов, например, в планировании задач.

28) **Связные динамические структуры. Стек.**

Стек — структура данных с принципом *LIFO* (последним пришёл — первым вышел). Реализуется через связный список или массив.

Основные операции: *push* — добавить элемент на вершину стека, *pop* — удалить верхний элемент и вернуть его.

Пример с односвязным списком:

```
struct Node {  
    int data;  
    Node * next;  
};
```

Место для уравнения. *Node * top = nullptr;*

```
Место для уравнения. void push(int val) {  
    Node * newNode = new Node{val, top};  
    top = newNode;  
}
```

```
Место для уравнения. int pop() {  
    if (!top) return -1; // или ошибка  
    int val = top->data;  
    Node * temp = top;  
    top = top->next;  
    delete temp;  
    return val;  
}
```

Стек используется в алгоритмах, вызовах функций и парсинге.

29) **Нелинейные динамические структуры. Деревья.**

Дерево — иерархическая структура, состоящая из узлов, где каждый узел может иметь несколько потомков.

Пример бинарного дерева:

```
struct Node {  
    int data;  
    Node * left;  
    Node * right;  
};
```

Используется для быстрого поиска, сортировки и представления иерархий.

30) Перегрузка функций.

Перегрузка функций — возможность объявлять несколько функций с одинаковым именем, но разными параметрами.

Пример:

```
int sum(int a, int b) { return a + b; }  
double sum(double a, double b) { return a + b; }
```

Компилятор выбирает функцию по типам и числу аргументов. Позволяет писать понятный и удобный код.

31) Перегрузка операций.

Перегрузка операций позволяет определить, как стандартные операторы (+, −, *, == и др.) работают с пользовательскими типами (например, структурами или классами).

Пример:

```
struct Point {  
    int x, y;  
    Point operator + (const Point& other) {  
        return {x + other.x, y + other.y};  
    }  
};
```

Теперь можно складывать объекты:

```
Point a{1, 2}, b{3, 4};  
Point c = a + b; // c = {4, 6}
```

Перегружать можно почти все операторы, кроме . (доступ к члену), :: (область видимости), ?: и *sizeof*.

32) Основные принципы объектно-ориентированного программирования.

Объектно-ориентированное программирование (ООП) основывается на 4 принципах:

1. Инкапсуляция — объединение данных и методов в классе, скрытие внутренней реализации.
2. Наследование — создание новых классов на основе существующих.
3. Полиморфизм — единый интерфейс для разных типов (одинаковые имена методов — разное поведение).
4. Абстракция — выделение существенного, скрытие деталей.

ООП помогает структурировать код, повышать читаемость и повторное использование.

33) Понятие класса. Инкапсуляция.

Класс — пользовательский тип, объединяющий данные и функции, которые с ними работают. Класс — основа ООП.

Пример:

```
class Rectangle {  
    private:  
        int width, height;  
    Место для уравнения. public:  
        void set(int w, int h) {  
            width = w;  
            height = h;  
        }  
        int area() {  
            return width * height;  
        }  
};
```

Инкапсуляция — сокрытие внутреннего состояния объекта и предоставление доступа через методы (например, *set*, *get*). Достигается через уровни доступа: *private*, *protected*, *public*.

34) Конструкторы и деструкторы.

Конструктор — специальная функция класса, вызываемая при создании объекта. Инициализирует поля.

Пример:

```
class MyClass {  
    public:  
        MyClass() {  
            // конструктор по умолчанию  
        }  
        MyClass(int x) {  
            // перегруженный конструктор  
        }  
};
```

Деструктор — вызывается при удалении объекта (в конце его жизни), освобождает ресурсы.

```
~MyClass() {  
    // очистка  
}
```

Если не определить явно, компилятор создаст их по умолчанию.

35) Наследование.

Наследование — механизм, позволяющий создавать новый класс на основе существующего.

Пример:

```
class Animal {  
    public:  
        void speak() { cout << "Some sound"; }  
};  
Место для уравнения. class Dog : public Animal {  
    public:  
        void bark() { cout << "Woof"; }  
};
```

Dog наследует все открытые (*public*) члены *Animal*. Существует: публичное наследование (*public*) — «является» (*Dog is an Animal*), защищённое (*protected*) — члены становятся защищёнными в потомке, Приватное (*private*) — члены становятся приватными. Наследование способствует повторному использованию кода и организации иерархий.

36) **Полиморфизм.**

Полиморфизм — способность одного интерфейса работать с разными типами данных. В C++ существует два вида:

- Компиляторный (перегрузка) — одна функция/оператор с разными типами аргументов.

- Виртуальный (во время выполнения) — используется при наследовании.

Базовый класс определяет виртуальные функции, производные классы реализуют свою версию.

Пример:

```
class Animal {  
public:  
    virtual void speak() { cout << "Animal sound"; }  
};
```

Место для уравнения. *class Dog : public Animal {*

```
public:  
    void speak() override { cout << "Woof"; }  
};
```

Место для уравнения. *Animal * a = new Dog();*

a->speak(); // Woof (вызовется Dog::speak)

Полиморфизм упрощает расширение программ и реализацию гибких интерфейсов.

37) **Структуризация. Восходящее программирование.**

Структуризация — процесс логического разбиения программы на части (модули, функции) для упрощения разработки и понимания.

Восходящее программирование (снизу вверх) — подход, при котором сначала разрабатываются и отлаживаются базовые компоненты (функции, модули), затем они объединяются в более сложные системы. Преимущества:

- 1) Повторное использование модулей.
- 2) Лучшая отладка и тестирование частей отдельно.
- 3) Программа строится как конструктор из готовых деталей.

38) Нисходящий метод проектирования программ.

Нисходящее программирование (сверху вниз) — сначала разрабатывается общая структура программы, затем она поэтапно уточняется. Шаги: определение основной задачи, деление на подзадачи, реализация каждой подзадачи отдельно.

Пример: написать игру → Разделить на модули (интерфейс, логика, ввод-вывод) → Реализовать каждый модуль.
Этот метод удобен для постановки задач и общей архитектуры, особенно на начальных этапах.

39) Методы тестирования программных средств, состоящих из одного модуля.

Для модулей применяются модульные тесты, проверяющие корректность отдельных функций и компонентов.

Методы:

1. Черный ящик — тестируем без знания внутренней реализации, только вход/выход.
2. Белый ящик — тестирование с анализом кода (покрытие условий, ветвей, путей).
3. Серый ящик — комбинация обоих.

Применяются утверждения (*assert*), автоматические фреймворки (Google Test, Catch2), ручная отладка. Цель — найти ошибки до интеграции с другими модулями.

40) Спецификация на программное средство.

Спецификация ПО — документ, в котором описано, что должно делать программное обеспечение. Это техническое задание, понятное как разработчику, так и заказчику.

Включает:

1. Назначение и функции программы.
2. Входные/выходные данные.
3. Ограничения и требования к среде.
4. Поведение в особых ситуациях (ошибки, сбои).
5. Интерфейс пользователя.

Хорошая спецификация — основа для проектирования, реализации и тестирования программы.

41) **Межмодульное тестирование программных средств.**

Межмодульное тестирование (интеграционное тестирование) — проверка взаимодействия нескольких модулей программы между собой. Цель — убедиться, что модули правильно обмениваются данными и вызывают друг друга.

Подходы:

1. Инкрементальный: поэтапное объединение модулей и тестирование каждого шага.
2. Нисходящий: сначала тестируется главный модуль, остальные заменяются заглушками.
3. Восходящий: сначала тестируются нижние модули, верхние заменяются драйверами.

Используются модули-заглушки, логирование, отладочные выводы, автоматические тесты.

42) **Основы доказательства правильности программ.**

Доказательство правильности программы — это строгое математическое подтверждение, что программа всегда работает правильно при всех допустимых входных данных.

Ключевые понятия:

1. Предусловие — условие, которое должно быть выполнено до выполнения программы.
2. Постусловие — ожидаемый результат после выполнения.
3. Инварианты — утверждения, сохраняющиеся при любом выполнении цикла/процедуры.

Пример (для цикла):

```
while (x < 10) {  
    x ++;  
}  
// инвариант: x <= 10
```

Методы:

- Логика предикатов.
- Аппроксимации (интерпретация программы как системы логических утверждений).
- Использование утверждений (*assert*).

43) **Жизненный цикл программы.**

Жизненный цикл программы — этапы существования ПО от идеи до завершения использования:

1. Постановка задачи — выявление требований.
2. Проектирование — архитектура, структура, выбор технологий.
3. Реализация — программирование.
4. Тестирование и отладка — устранение ошибок.
5. Внедрение — установка и запуск у пользователя.
6. Эксплуатация и сопровождение — обновления, поддержка.
7. Вывод из эксплуатации — замена, утилизация.

Управление жизненным циклом — основа качественной разработки программных продуктов.

44) **Причины отказов программного обеспечения (ПО) и повышение отказоустойчивости ПО к ошибкам.**

Причины отказов ПО:

- Программные ошибки (баги).
- Неправильные требования.
- Ошибки проектирования.
- Влияние внешней среды (перегрузка, сбой оборудования).

Повышение отказоустойчивости:

- Обработка исключений (*try – catch*).
- Проверка входных данных.
- Автоматическое восстановление состояния.
- Резервирование данных (бэкапы).
- Юнит-тесты и тестирование на отказ.

Отказоустойчивость особенно важна в критических системах (медицина, транспорт, финансы).

45) **Направления в программировании. Классификация языков программирования.**

Направления в программировании:

- Системное (работа с ОС, драйверами) — C, C++.
- Прикладное (офисные приложения, игры) — C++, Java, Python.
- Веб-программирование — HTML, CSS, JavaScript, PHP.
- Научное и численное — Fortran, Python, MATLAB.
- Мобильное — Kotlin, Swift.
- Искусственный интеллект — Python, R.

Классификация языков:

- По уровню: низкоуровневые (Assembler), высокоуровневые (C++, Python).
- По парадигме:
 1. процедурные (C),
 2. объектно-ориентированные (C++, Java),
 3. функциональные (Haskell, Lisp),
 4. логические (Prolog).
- По назначению: общего назначения (C++), специализированные (SQL, HTML).

46) **Критерии качества программ.**

Качество программного обеспечения определяется рядом характеристик:

1. Надежность — устойчивость к ошибкам, корректная работа.
2. Эффективность — быстрое выполнение, экономия ресурсов.
3. Удобство использования (юзабилити) — понятный интерфейс.
4. Масштабируемость — возможность расширения и модификации.
5. Тестируемость — легкость обнаружения и устранения ошибок.
6. Переносимость — возможность работы на разных платформах.
7. Безопасность — защита от внешних угроз.
8. Документированность — наличие комментариев, инструкций, спецификаций.

Пример: программа, потребляющая минимум памяти, с простым интерфейсом и понятной структурой — качественная по большинству критериев.

47) **Директивы препроцессора. Директива `define`, макроподстановки.**

Препроцессор — стадия обработки кода до компиляции. Основные директивы:

- `#define` — определяет макросы (замены текста):

`#define PI 3.14`

`#define MAX(a,b) ((a) > (b) ? (a) : (b))`

- `#include` — подключение заголовочных файлов:

`#include <iostream>`

`#include "custom.h"`

- `#ifdef`, `#ifndef`, `#endif` — условная компиляция:

`#ifdef DEBUG`

`cout << "Debug mode";`

`#endif`

Макросы — текстовые замены, не проверяются компилятором, поэтому могут вызывать ошибки, если неаккуратно использовать (например, без скобок).

48) **Модульное программирование. Структура программы, состоящей из нескольких файлов.**

Модульное программирование — подход, при котором программа делится на независимые части (модули), каждый из которых выполняет определённую функцию.

Пример структуры:

`main.cpp` // главный файл

`math.cpp, math.h` // реализация и интерфейс модуля `math`

`io.cpp, io.h` // ввод — вывод

Преимущества:

- Упрощение отладки и тестирования.
- Повторное использование модулей.
- Разделение труда между разработчиками.

Компиляция: каждый `.cpp` компилируется отдельно, затем связывается (линковка).

49) **ГОСТ 19.701. Виды схем, символов. Обозначения для схем алгоритмов и схем работы системы.**

ГОСТ 19.701–90 – стандарт на схемы алгоритмов, используется при проектировании ПО (особенно в государственных организациях).

Виды схем:

- Функциональная схема – отражает работу системы.
- Алгоритмическая схема – последовательность действий (алгоритм).
- Структурная схема – архитектура системы.

Основные символы:

- Овал – начало/конец.
- Прямоугольник – операция.
- Ромб – условие.
- Параллелограмм – ввод/вывод.
- Прямоугольник с отделёнными краями – вызов подпрограммы.
- Шестиугольник – цикл.
- Круг – ссылка.